Volume 12 – DevOps, Infrastructure & Cloud Architecture

AI Quant Trading Platform

Version: 1.0

Document Type: DevOps & Infrastructure Design

Classification: Production Infrastructure

---

1. Purpose

This document defines the production infrastructure, cloud architecture, deployment strategy, CI/CD pipeline, monitoring, disaster recovery, security hardening, and operational standards for the AI Quant Trading Platform.

The infrastructure is designed to support:

- 24×7 automated trading
- High availability
- Horizontal scalability
- Low-latency execution
- Disaster recovery
- Secure operations
- Continuous deployment

---

2. Infrastructure Goals

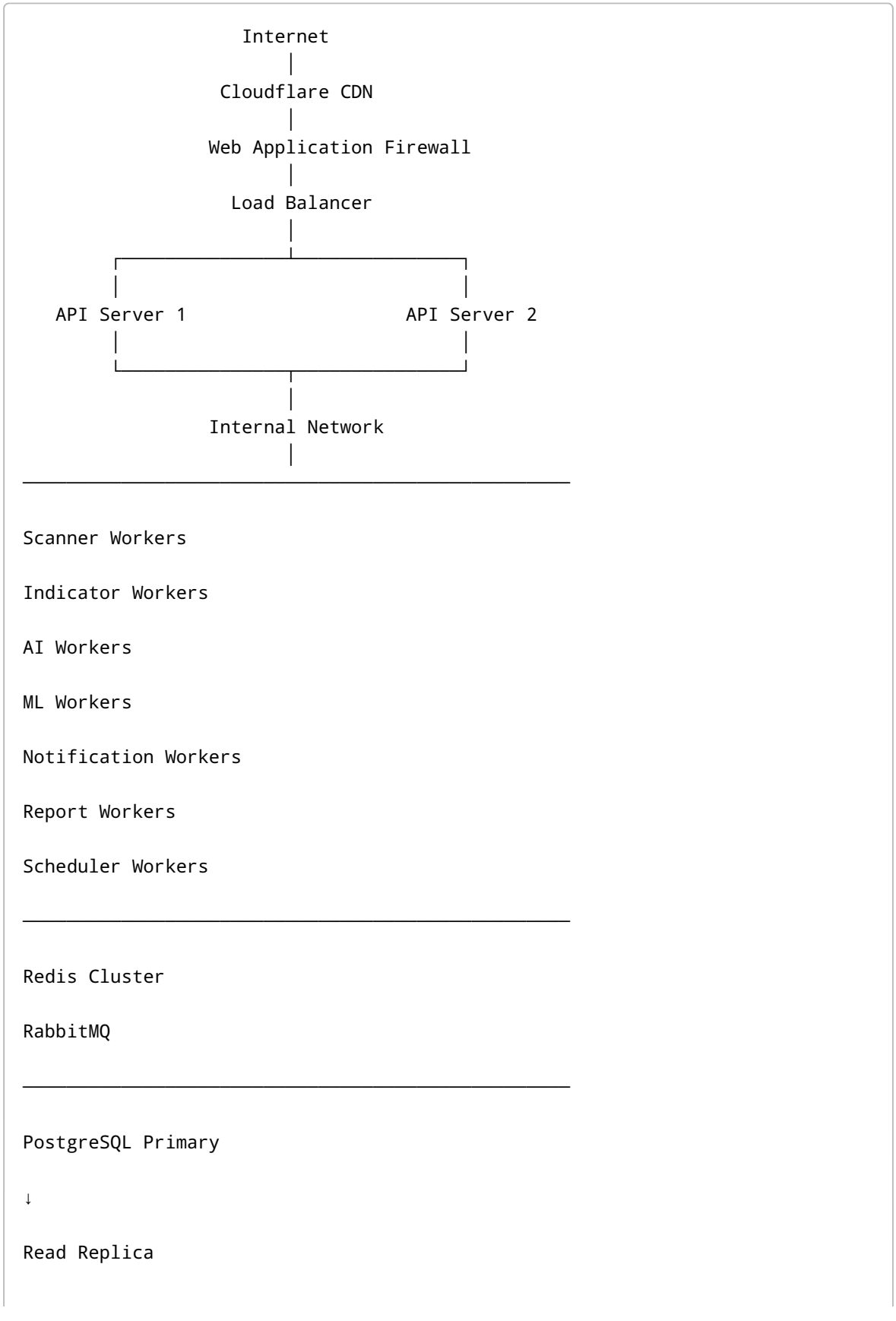
Primary Goals

- Zero-downtime deployments
- Automatic recovery
- Horizontal scaling
- Infrastructure as Code
- Secure production environment
- Continuous monitoring
- High performance

Target Availability

- 99.9% minimum
- 99.99% preferred

3. High-Level Infrastructure



```
↓

Backup Storage

──────────────────────────────────────────────────

Monitoring

Logging

Alerting

──────────────────────────────────────────────────

Exchange APIs
```

---

4. Production Stack

Operating System

- Ubuntu LTS Server

Container Runtime

- Docker

Container Orchestration

- Docker Compose
- Kubernetes (future)

Reverse Proxy

- Nginx

Application Servers

- Gunicorn
- Uvicorn

Database

- PostgreSQL

Cache

- Redis

Message Queue

- RabbitMQ

---

5. Docker Architecture

Containers

- API
- FastAPI
- Scanner
- AI
- ML
- Celery Worker
- Celery Beat
- Redis
- PostgreSQL
- RabbitMQ
- Nginx
- Prometheus
- Grafana
- Loki
- MinIO (optional)

Each service runs independently.

---

6. Environment Strategy

Development

Local machine

Testing

Shared environment

Staging

Production replica

Production

High Availability Cluster

Each environment uses isolated configuration and credentials.

---

7. CI/CD Pipeline

Developer

↓

Git Repository

↓

Automated Testing

↓

Code Quality Checks

↓

Security Scan

↓

Build Docker Images

↓

Push to Registry

↓

Deploy to Staging

↓

Smoke Tests

↓

Manual Approval

↓

Production Deployment

---

8. Branch Strategy

main

Production-ready code

develop

Integration branch

feature/*

Feature development

hotfix/*

Emergency fixes

release/*

Release preparation

---

9. Infrastructure as Code

Recommended

- Terraform
- Ansible

Manage

- Servers
- Networking
- DNS
- Firewall
- Storage
- Secrets
- Monitoring

---

10. Monitoring

Infrastructure Metrics

- CPU
- RAM

- Disk
- Network

Application Metrics

- API Latency
- WebSocket Connections
- Scanner Throughput
- Order Processing Time
- Queue Length
- Worker Health

Database Metrics

- Active Connections
- Query Time
- Replication Lag

---

11. Logging

Centralized logging.

Every service logs:

- Timestamp
- Request ID
- User ID
- Tenant ID
- Service Name
- Severity
- Message
- Stack Trace (if applicable)

Recommended Stack

- Loki
- Elasticsearch
- Kibana

---

12. Alerting

Alerts for:

- API Failure
- High CPU
- High Memory
- Database Down

- Redis Down
- Exchange API Failure
- Queue Backlog
- Worker Failure
- High Error Rate
- Disk Space

Notifications

- Email
- Telegram
- Slack
- PagerDuty (future)

---

13. Backup Strategy

Database

- Daily Incremental
- Weekly Full

Configuration

- Daily

Object Storage

- Daily

Logs

- Configurable retention

Backups must be encrypted and periodically restored in a test environment to verify integrity.

---

14. Disaster Recovery

Recovery Objectives

- Automated restart where possible
- Database restoration
- Configuration restoration
- Queue recovery
- Service health validation

Maintain documented recovery procedures and test them regularly.

---

15. Security Hardening

Servers

- SSH key authentication
- Disable password login
- Firewall enabled
- Automatic security updates
- Fail2Ban

Containers

- Non-root users
- Read-only filesystem where practical
- Image vulnerability scanning
- Resource limits

---

16. Secrets Management

Never store secrets in source code.

Secrets include:

- Exchange API Keys
- JWT Secrets
- Database Passwords
- Redis Passwords
- SMTP Credentials

Recommended

- Environment variables
- Dedicated secrets manager in production

---

17. Network Design

Public Network

- Nginx
- Load Balancer

Private Network

- API Services
- Workers
- Database

- Redis
- RabbitMQ

Databases and queues must not be directly accessible from the public internet.

---

18. Scaling Strategy

Horizontal Scaling

- API Servers
- Scanner Workers
- AI Workers
- Notification Workers

Vertical Scaling

- Database
- Redis

Automatic scaling policies may be introduced in future releases.

---

19. High Availability

Redundant API servers

Database replication

Redis persistence

Health checks

Automatic service restart

Graceful shutdown

---

20. Performance Targets

API Response

<300 ms (typical)

Market Updates

Near real-time

Scanner Cycle

Configurable based on exchange limits

Background Workers

Process jobs asynchronously without blocking user requests.

---

21. Maintenance

Scheduled maintenance includes:

- Database optimization
- Log cleanup
- Image updates
- Security patching
- Backup verification
- Certificate renewal
- Dependency updates

Maintenance windows should be announced in advance.

---

22. Deployment Checklist

Before Production

- Tests Passed
- Security Scan Passed
- Database Migration Reviewed
- Backup Created
- Monitoring Enabled
- Alerts Configured
- Rollback Plan Prepared
- Documentation Updated

---

23. Future Enhancements

- Kubernetes
- Multi-region deployment
- Auto Scaling
- GPU inference nodes
- Service Mesh
- Multi-cloud support
- Edge caching

- Blue/Green deployments
- Canary releases

---

24. Operational Principles

- Infrastructure should be reproducible.
- Every deployment must be reversible.
- Monitoring is mandatory.
- Backups are verified, not just created.
- Security is integrated into every deployment stage.
- Critical services should fail safely and recover automatically where possible.